Documentation de l'extension

Projet Génie Logiciel

Groupe 1, gl06

ZIVKOVIC Lucas, SULLET Benjamin, ZOUGGARI Samy,
TRAORÉ Fié Victor, HURSTEL Mathis

20 Janvier 2025

Grenoble INP - Ensimag
Génie Logiciel 2024-2025

Sommaire

1 Introduction **2**

2 Spécification et analyse bibliographique **3**
2.1 Déclaration des tableaux 3
2.2 Types des tableaux 4
2.3 Longueur des tableaux 5

3 Choix de conception, d'architecture et d'algorithmes **5**
3.1 Analyse lexicale 5
3.2 Analyse grammaticale 5
3.2.1 Modifications des règles existantes 7
3.2.2 Création des noeuds 9
3.3 Analyse contextuelle 12
3.3.1 Modification des vérifications contextuelles existantes . 12
3.3.2 Nouvelles vérifications contextuelles 14

4 Génération de code **17**
4.1 Modification de méthodes existantes 18
4.2 Implémentation de nouvelles méthodes 20

5 Bibliothèque de calcul matriciel **22**
5.1 Description des fonctionnalités 22
5.2 Guide d'utilisation 27

6 Méthodes et résultats de validation **28**
6.1 Méthodes de validation 28
6.2 Résultats de validation 28

7 Références bibliographiques **29**

1 Introduction

L'extension `TAB` vise à doter le langage Deca d'une gestion complète des tableaux. De base, Deca ne propose pas de structures pour manipuler des tableaux statiques de façon systématique ; cette extension permettra d'adresser ce besoin et d'étendre le potentiel du langage. Concrètement, il s'agit de rendre possibles les déclarations de tableaux unidimensionnels et multidimensionnels, leur initialisation (ex. `{1, 2, 3}`) et leurs différentes manipulations (accès indexé, opérations entre tableaux, etc.).

Pour ce faire, nous devons étendre chacune des étapes principales de la compilation Deca :

- **Analyse lexicale (Etape A)** : introduire les nouveaux symboles nécessaires aux tableaux (par ex. les crochets `[ ]` pour déclarer et accéder aux indices).
- **Analyse syntaxique (Etape B)** : enrichir la grammaire afin de gérer la déclaration d'un tableau, sa taille, son initialisation, et la syntaxe d'accès indexé (ex. `tab[i]`).
- **Analyse contextuelle (Etape C)** : vérifier la cohérence de types (ex. un `float[]` ne peut contenir que des nombres flottants ou `int` convertis), la validité des dimensions (le nombre de crochets doit correspondre à la dimension attendue) et le fait que chaque indice soit bien un entier.
- **Génération de code** : produire les instructions permettant l'allocation mémoire des tableaux et la gestion des index (calcul d'offset, boucles d'initialisation, etc.).

En plus de ces modifications, l'extension propose une bibliothèque de calcul matriciel permettant de mettre à profit les nouveaux tableaux : opérations élémentaires (somme, soustraction, etc.), calculs plus avancés (produit scalaire, tri ...). L'ambition est de doter Deca d'un environnement plus riche, où la manipulation de données en tableau devient aussi simple qu'intuitive. Ainsi, l'utilisateur pourra non seulement déclarer et parcourir des tableaux, mais aussi s'appuyer sur des fonctions prêtes à l'emploi pour mener des traitements mathématiques ou algorithmiques plus élaborés.

Remarque : contrairement a ce qui est mentionné dans le manuel utilisateur, il n'est pas nécessaire d'utiliser l'extension `-e` pour compiler un programme `.deca` nécessitant d'utiliser l'extension.

2 Spécification et analyse bibliographique

Pour implémenter les tableaux dans Deca, nous nous inspirons fortement de leur représentation en Java.

2.1 Déclaration des tableaux

Les tableaux en Java peuvent être déclarés de plusieurs manières :

Déclaration sans initialisation Un tableau peut être déclaré sans préciser ses valeurs initiales. La syntaxe utilisée est la suivante :

```
new dataType[length_1]...[length_n]
```

où :

- `dataType` correspond au type des éléments du tableau,
- chaque paire de crochets `[]` représente une dimension du tableau,
- les valeurs `length_i` définissent la taille des sous-tableaux pour chaque dimension.

Exemple :

```
int[][] array = new int[2][3];
```

Ce code crée un tableau à deux dimensions (2 lignes, 3 colonnes). Nous pouvons le voir comme si le tableau était constitué de deux sous-tableau de taille 3.

Déclaration avec initialisation des valeurs Les tableaux peuvent aussi être initialisés directement avec leurs valeurs. Cela peut se faire de deux façons différentes :

- **Initialisation implicite :** le type du tableau est déduit automatiquement.

  ```
  int[] array = {1, 5, 4, 3, 7};
  int[][] matrix = {{1, 2}, {4, 5}, {7, 8}};
  ```

  Cette méthode permet de spécifier les valeurs sans déclarer explicitement le type.

- **Initialisation explicite :** on utilise le mot-clé `new`. Le type du tableau est précisé explicitement ensuite.

```
int[] array = new int[]{1, 5, 4, 3, 7};
int[][] matrix = new int[][]{{1, 2, 3},
{4, 5, 6}, {7, 8, 9}};
```

Ici, les valeurs sont initialisées et le type du tableau est explicitement indiqué.

2.2 Types des tableaux

En Java, le type d'un tableau est défini par un type de base suivi d'un certain nombre de `[]` indiquant la dimension :

```
int[][][] x;
```

Ce code déclare un tableau d'entiers à trois dimensions.

Remarque :

- Il est également possible de combiner les crochets dans la déclaration du type et de la variable lors de l'initialisation d'un tableau. Par exemple :

  ```
  int[] x[];
  ```

  Cette déclaration est équivalente à :

  ```
  int[][] x;
  ```

  Dans les deux cas, `x` est un tableau à deux dimensions.

- Il est donc possible de déclarer un tableau de n'importe quel type sous condition qu'il soit déclaré dans l'environnement des types. Par exemple, nous pouvons écrire :

  ```
  class A{}
  {
      A[] x;
  }
  ```

  est possible car la classe A a été déclarée. Le type de A est donc disponible comme base pour construire de nouveaux types.

Contenu des tableaux Les tableaux peuvent contenir des expressions complexes, pas uniquement des littéraux. Par exemple :

```
int[][] x = {{new int[]{1, 2}}, {new int[]{3, 4}}};
```

Ici, chaque sous-tableau est créé avec des expressions valides.

2.3 Longueur des tableaux

Il est possible d'accéder à la longueur d'un tableau déclaré en Java grâce au caractère `length`. Par exemple, le script suivant :

```
{
    int[] x = new int[5];
    println(x.length);
}
```

renvoie la valeur 5. Il est possible d'accéder à la longueur d'un sous-tableau en écrivant :

```
{
    int[][] x = new int[5][3];
    println(x[0].length);
}
```

Ceci renvoie la valeur 3 qui est la longueur du premier sous-tableau (et qui est aussi la taille de tous les sous-tableaux).

3 Choix de conception, d'architecture et d'algorithmes

3.1 Analyse lexicale

Pour prendre en charge les tableaux, nous avons ajouté les tokens suivants dans l'analyse lexicale :

- `OCROCHET` : correspond au caractère `[`,
- `CCROCHET` : correspond au caractère `]`.
- `LENGTH` : correspond au caractère `length`.

Ces tokens sont essentiels pour reconnaître la structure des tableaux dans les prochaines étapes.

3.2 Analyse grammaticale

Plusieurs règles ont été ajoutées ou modifiées dans le parser pour intégrer les tableaux :

`array_literal` : Cette règle génère une instance Java `ArrayLiteral` qui contient le texte brut d'un tableau entouré d'accolades (par exemple : {`1, 2, 3`}). Elle suit une logique similaire à la règle `literal` (voir figure 1). De plus, elle récupère les expressions entre les accolades pour construire les coefficients du tableau.

```
array_literal returns [AbstractExpr tree,  List<ExprContext> exp]
    : OBRACE (exp_list+=expr (COMMA exp_list+=expr)*)? CBRACE {
        $exp = $exp_list;
        $tree = new ArrayLiteral($ctx.getText(), $exp_list);
        setLocation($tree, $OBRACE);
    }
    ;
```

Figure 1: Implémentation de la nouvelle règle `array_literal`

`array` : Cette règle permet l'instanciation de tableaux via la syntaxe `new` .... Elle prend en charge à la fois :

- Les tableaux vides : `new int[][]`.
- Les tableaux initialisés : `new int[]{1, 2}`.

Cette règle utilise un appel à `array_literal` pour gérer l'initialisation des valeurs dans le cas échéant

```
array returns[AbstractExpr tree]
    : IDENT (OCROCHET indices+=expr CCROCHET)+ {
        $tree = new ArrayNoInit($ctx.getText(), $indices);
        setLocation($tree, $IDENT);
    }
    | IDENT (OCROCHET CCROCHET)+ array_literal {
        $tree = new ArrayLiteral($ctx.getText(), $array_literal.exp);
        setLocation($tree, $IDENT);
    }
    ;
```

Figure 2: Implémentation de la nouvelle règle `array`

`get_array` Cette règle est utilisée pour reconnaître l'accès à des tableaux avec des expressions du type :

```
int[][] x = new int[2][2];
x[0][0] = 1;
```

Cette règle est utilisée aussi bien dans le cadre d'un "get" que d'un "set". La différence se situe dans le fait que dans un "get", l'expression d'accès à un tableau est à droite de l'égalité. Alors que dans un set, l'expression d'accès à un tableau se situe à gauche de l'égalité. Ainsi, savoir si on fait un "get" ou un "set" n'est pas utile au cours de cette étape.

```
get_array returns[AbstractExpr tree]
    :ident (OCROCHET list_int+=expr CCROCHET)+ {
          $tree = new GetArray($ident.tree, $list_int);
          setLocation($tree, $ident.start);
    }
    ;
```

Figure 3: Implémentation de la nouvelle règle `get_array`

3.2.1 Modifications des règles existantes

Certaines règles déjà présentes ont été ajustées pour prendre en charge les tableaux :

- `ident` : désormais, cette règle reconnaît une séquence de tokens `IDENT (OCROCHET CCROCHET)*`, permettant ainsi de gérer les identifiants des tableaux.

```
ident returns[AbstractIdentifier tree]
    :
     IDENT (OCROCHET CCROCHET)* {
        $tree = new Identifier(getDecacCompiler().createSymbol($ctx.getText()));
        setLocation($tree,$IDENT);
        }
    ;
```

Figure 4: Nouvelle implémentation de la règle `ident`. De base il n'ya seulement le token `IDENT`

- `literal` : ajout du traitement des tableaux via `array_literal`, en considérant les tableaux comme des littéraux.

```
| THIS {
    $tree = new This();
    setLocation($tree,$THIS);
    }
| NULL {
    $tree = new Null();
    setLocation($tree,$NULL);
    }
| array_literal {
    assert($array_literal.tree != null);
    $tree= $array_literal.tree;
}
;
```

Figure 5: ajout de la dérivation vers la règle `array_literal` à la fin de la règle `primary_expr`

- `primary_expr` :
  - ajout de la règle `NEW array` pour reconnaître des initialisations de tableaux comme des expressions valides.
  - ajout de la règle `get_array` pour reconnaître les expressions d'accès à des tableaux.
  - ajout de la règle `ident DOT LENGTH` pour reconnaître les expressions de longueurs d'un tableau.

```
| literal {
        assert($literal.tree != null);
        $tree=$literal.tree;
}
| NEW array {
              assert($array.tree !=null);
              $tree=$array.tree;
              setLocation($tree, $NEW);
}

| get_array {
        assert($get_array.tree != null);
        $tree=$get_array.tree;
}
| ident DOT LENGTH {
        assert($ident.tree != null);
        $tree= new LengthArray($ident.tree);
        setLocation($tree, $ident.start);
}
;
```

Figure 6: Ajout des dérivations dans la règle `primary_expr`

3.2.2 Création des noeuds

De plus, plusieurs classes correspondantes aux noeuds de l'arbre grammatical ont été créées.

- `ArrayType` :

  Cette classe hérite de la classe mère `Type`. Elle permet d'attribuer un type spécifique `ArrayType` à une variable, avec deux attributs principaux :

- `dataType` (`Type`) : représente le type des éléments du tableau.
- `dimension` (`int`) : indique le nombre de dimensions du tableau.

La création d'une instance de cette classe intervient lors de la vérification contextuelle. Le symbole ajouté à l'environnement des types correspond directement à celui utilisé par l'utilisateur. Par exemple, pour la déclaration suivante :

```
int[] x;
```

le symbole `int[]` est ajouté à l'environnement des types.

Cette classe apporte des spécifications spécifiques aux tableaux, notamment :

- Le type des éléments qu'ils contiennent (par exemple `int`, `float`, etc.).
- La dimension du tableau.

Cette classe redéfinit la méthode `sameType` de `Type` en vérifiant que les dataType et les dimensions correspondent.

Rôle principal : S'assurer que les tableaux respectent les contraintes de type. Par exemple, un tableau défini comme `ArrayType[int, 2]` doit être un tableau à deux dimensions contenant uniquement des entiers.

Remarque : la création de cette classe implique la création d'une méthode `isArray` dans la classe `Type` qui est redéfinie ici en suivant le même principe que les autres méthodes au comportement similaire.

- `ArrayLiteral` :

  Une instance de cette classe est créée par le parser. Cette classe, qui hérite de `AbstractExpr`, possède un constructeur avec les paramètres suivants :

  - Un paramètre de type `String` : récupère le texte brut correspondant à la déclaration d'un tableau.
  - Un paramètre de type `List<DecaParser.ExprContext>` : récupère les coefficients sous forme d'expressions issues de la règle de parser `expr`.

  Cette classe effectue les traitements nécessaires pour :

  - Déterminer la dimension du tableau.

- Générer la liste des tailles respectives du tableau et de ses sous-tableaux.
- Créer une instance de la classe `Array`.

Au final, cette classe possède les attributs suivants :

- `name` (`String`) : représente le nom brut du tableau tel qu'il a été écrit par l'utilisateur, sans modifications.
- `dimension` (`int`) : deux méthodes permettent de déterminer la dimension du tableau, selon qu'il a été initialisé ou non.
- `sizes` (`List<Integer>`) : correspond à la liste des tailles respectives du tableau et de ses sous-tableaux, jusqu'à atteindre les sous-tableaux de dimension 1.
- `array` (`Array`) : représente une instance de la classe `Array`, présentée ci-dessous.

- `Array` :

  Cette classe, qui hérite également de `AbstractExpr`, est instanciée par `ArrayLiteral`. Elle possède les attributs suivants :

  - `size` (`int`) : représente la taille directe du tableau, sans prendre en compte la taille de ses sous-tableaux.
  - `coefficients` (`AbstractExpr[]`) : un tableau contenant les coefficients récupérés par la classe `ArrayLiteral`, traités et stockés sous forme d'expressions abstraites.

- `ArrayNoInit` : Cette classe est instanciée, comme son nom l'indique, pour la création de tableaux sans initialisation de leurs coefficients. En effet, il existe un certain nombre de traitements contextuels (dimensions, tailles, etc.) qui diffèrent selon que le tableau est déclaré avec ou sans initialisation. Afin de gérer ces spécificités de manière claire, nous avons décidé de séparer en deux classes distinctes la déclaration de tableaux avec initialisation et celle de tableaux sans initialisation.

  - `name` (`String`) : Stocke le nom du tableau sous la forme brute, c'est-à-dire exactement tel que saisi par l'utilisateur.
  - `indices` (`List<AbstractExpr>`) : Liste des expressions correspondant aux indices, définissant la (ou les) dimension(s) du tableau.

- `ArrayDefinition` :

  Cette classe hérite de `VariableDefinition` et possède un unique attribut :

  - `init` : représente l'initialisation d'un identificateur.

  Elle étend la classe mère et est utilisée pour récupérer le tableau associé à un identificateur défini. Cela permet d'effectuer des vérifications contextuelles sur ce tableau lorsqu'il n'est pas directement accessible, par exemple lors de l'appel de l'identificateur correspondant.

- `GetArray` :

  Cette classe étend la classe `Identifier`. Il est nécessaire qu'elle hérite de `Identifier` pour pouvoir être utilisée en tant que `Lvalue`. En effet, les expressions comme :

  ```
  x[0] = 1;
  ```

  exigent que le terme à gauche soit une instance de `Identifier`.

  Cette classe possède les attributs suivants :

  - `identifier` : (`AbstractIdentifier`) correspond à l'identificateur auquel on souhaite accéder pour obtenir un coefficient.
  - `listIndex` : (`List<IntLiteral>`) représente la liste des indices permettant d'accéder au coefficient souhaité.

  Par exemple, l'expression suivante :

  ```
  x[0][1][2] = 1;
  ```

  crée une instance de `GetArray` ayant comme attributs l'`identifier x` et la liste d'indices `(0, 1, 2)`.

- `LengthArray` : Une instance de cette classe est créée lors de la saisie d'une expression du type `x.length` avec `x` un caractère considéré comme un identifiant défini au préalable. Cette classe possède un unique attribut `ident` de type `Identifier`.

D'autres noeuds seront modifiés dans le cadre de l'analyse contextuelle.

3.3 Analyse contextuelle

3.3.1 Modification des vérifications contextuelles existantes

L'implémentation de l'extension **TAB** nécessite une mise à jour de certaines méthodes de vérification contextuelle déjà en place :

Mise à jour de la méthode `Identifier` La méthode de vérification contextuelle de `Identifier` est incomplète. De plus, cette méthode (`verifyType`) est appelée par `DeclVar`, ce qui implique qu'elle doit être capable de :

- Créer le type correspondant à l'identificateur.
- Retourner ce type pour permettre la construction correcte des types de tableaux (et les types intermédiaires).

Pour prendre en compte les nouveaux types tableaux, la nouvelle vérification contextuelle doit suivre les étapes suivantes :

1. Vérifier si le type de base (sans les crochets) existe dans l'environnement des types.
2. Déterminer le nombre de paires de crochets `[]` présentes dans la déclaration de la variable.
3. Construire progressivement les types intermédiaires jusqu'à atteindre le type final de la variable déclarée.

Mise à jour de la méthode `DeclVar` Comme pour la classe Identifier, la méthode de vérification contextuelle de la classe `DeclVar` est actuellement incomplète pour gérer les tableaux. En effet, l'environnement des types de base ne reconnaît pas directement les différents types associés aux tableaux.

Le processus de création de type est similaire à celui de `Identifier`, à la différence que `Identifier` ne doit pas générer de types intermédiaires inutiles, afin d'éviter toute redondance.

Mise à jour de la méthode `Assign` Pour les tableaux implicites (définis uniquement par des crochets), il est difficile de leur attribuer un type directement en fonction de leur valeur brute. En effet, plusieurs complications peuvent survenir :

- Le tableau peut contenir des erreurs (incohérences au niveau des types).
- Il est impossible de déterminer avec certitude le type de base en se basant uniquement sur un élément du tableau.

C'est pourquoi, lorsqu'une variable est assignée à un tableau implicite, nous avons décidé d'adopter l'approche suivante :

1. Définir le type du tableau implicite comme étant celui de la variable assignée.

2. Effectuer ensuite les vérifications contextuelles sur les éléments du tableau afin de s'assurer que leurs types sont cohérents avec celui de la variable assignée.

Exemple illustratif de vérifications contextuelles : Nous nous attardons sur l'illustration de la nécéssité de modifier les méthodes de vérification contextuelles de Identifier et de DeclVar. Considérons la déclaration suivante, sans initialisation :

```
int[][] x[] = {{{1,2},{3,4}},{{5,6},{7,8}}};
```

1. Lors de l'analyse contextuelle, `DeclVar` appelle `verifyType` pour l'identificateur `int[][]`.

2. `verifyType` renvoie alors le type `int[][]`. Pour cela, il est nécessaire que ce type soit déjà présent dans l'environnement des types du compilateur.

3. Cependant, la variable `x` est déclarée comme un tableau à trois dimensions (`int[][][]`). Il faut donc compléter la construction du type en ajoutant les dimensions restantes dans `verifyType` de `DeclVar`. Ici, comme `int[]` et `int[][]` sont déjà déclarés. Il reste le type `int[][][]` à créer dans cette méthode.

Ensuite, nous passons par la méthode `verifyExpr` de la classe `Assign`. Celle-ci adresse à l'expression de droite le type `int[][][]`, vérifie que chaque sous-tableau direct est de dimension `int[][]`, etc. Arrivé au niveau terminal, on vérifie que chaque coefficient est de type `int`. Cette étape de vérification est détaillée dans la partie suivante.
En appliquant cette approche, la vérification contextuelle garantit une gestion correcte des types de tableaux dans l'extension **TAB**.

3.3.2 Nouvelles vérifications contextuelles

- `ArrayLiteral` : La classe `ArrayLiteral` est responsable de représenter les déclarations et initialisations de tableaux dans le cadre de l'analyse syntaxique et contextuelle. Elle s'assure que les tableaux définis respectent les règles de type et de dimension imposées par le langage. Voici les étapes principales des vérifications contextuelles réalisées dans cette classe :

1. **Identification de l'initialisation**
   La méthode `thereIsInit` détermine si un tableau est explicitement initialisé lors de sa déclaration. Par exemple, une déclaration comme :

   ```
   int[][] array = {{1, 2}, {3, 4}};
   ```

   est considérée comme initialisée. Cette distinction influence la manière dont les dimensions et les tailles sont calculées :

   - **Non initialisé** : Les dimensions sont extraites à partir des crochets `[]` dans le nom.
   - **Initialisé** : Les dimensions sont extraites à partir des accolades {} entourant les valeurs.

2. **Vérification de l'existence du type**
   Lors de l'analyse contextuelle, la méthode `verifyExpr` vérifie que le type de base du tableau est défini dans l'environnement des types. Par exemple, pour un tableau déclaré comme `int[]`, la méthode s'assure que le type `int` est connu. Si ce n'est pas le cas, une exception `ContextualError` est levée avec un message précisant que le type est inexistant.

3. **Correspondance des dimensions**
   Si un tableau est explicitement initialisé, la méthode vérifie que le nombre de dimensions spécifiées correspond à la structure des données. Cette vérification est effectuée en comparant la dimension extraite du nom du tableau avec celle définie lors de l'assignation ou la déclaration.
   Par exemple :

   ```
   int[][] array = {{1, 2}, {3, 4}};
   ```

   - La méthode s'assure que le type est `ArrayType[int, 2]`.
   - Si les dimensions ne correspondent pas, une exception est levée.

4. **Traitement des tailles des sous-tableaux**
   La méthode `setSizeFromNameInit` calcule les tailles maximales des sous-tableaux à chaque profondeur. Cette étape est essentielle pour valider les structures imbriquées et garantir leur cohérence.
   Par exemple :

   ```
   int[][] array = {{1, 2}, {3, 4, 5}};
   ```

- Cette méthode identifie que le premier sous-tableau contient 2 éléments, tandis que le second en contient 3.
- Elle enregistre ces tailles dans l'attribut `sizes`.

5. **Association des types aux coefficients**
   La méthode `array.verifyExpr` est appelée pour vérifier que chaque coefficient du tableau est du type attendu. Cela garantit que tous les éléments respectent les contraintes définies dans l'environnement des types.

- `Array` :
  - Cette classe permet de parcourir et de valider récursivement les éléments d'un tableau. Elle prend en charge les tableaux imbriqués.
  - Chaque élément du tableau est une instance d'`AbstractExpr`, qui peut être :
    * Un `literal` simple, comme un `int` ou un `float`.
    * Un tableau imbriqué, avec une dimension inférieure au tableau actuel (dim-1).
  - La validation est effectuée récursivement:
    * Si un élément est un `literal`, son type est comparé au type attendu.
    * Si un élément est un tableau, la validation est relancée sur ce sous-tableau, en prenant en compte son type et sa dimension.
  - **Exemple détaillé** :
    * Supposons un tableau `{{1, 2}, {3, 4}}`, qui doit être de type `ArrayType[int, 2]`.
    * La validation vérifie chaque niveau du tableau :
      · Au premier niveau, le tableau a bien deux sous-tableaux de dimension $n-1$ (ici 1), ce qui est conforme à un tableau dimension de 2.
      · Au second niveau, chaque élément des sous-tableaux est un entier (`int`), ce qui est conforme au type des éléments.
    * Si l'un des sous-tableaux avait une dimension différente de 1 ou contenait un élément d'un autre type, une erreur contextuelle de type serait signalée.

- `GetArray` :
  Les vérifications contextuelles réalisées par `GetArray` s'effectuent essentiellement dans la méthode `verifyExpr`. Elles assurent que l'accès à un élément (ou coefficient) de tableau est conforme aux règles de cohérence de type et de dimensions. Voici les principales étapes :

  1. **Validité des indices**
     Le constructeur de `GetArray` impose que tous les indices soient de type `int`.
  2. **Vérification de la déclaration du symbole**
     La méthode `verifyExpr` récupère l'identificateur du tableau (`identifier`) dans l'environnement courant (`localEnv`). Si le symbole n'y existe pas, une erreur est levée, indiquant que l'identificateur est inconnu.
  3. **Contrôle du type `ArrayType`**
     Une fois l'identificateur trouvé, `GetArray` vérifie que l'identificateur s'agit bien d'un `ArrayType`. Si ce n'est pas le cas, l'accès est invalide et une `ContextualError` est déclenchée.
  4. **Calcul du type après accès**
     Les indices (`listIndex`) déterminent la profondeur d'accès dans le tableau. `GetArray` adapte le type en retirant les niveaux de `[]` pour obtenir le type final :
     - Si toutes les dimensions sont parcourues, le type final peut être un type de base (`int`, `float`, etc.), ou un `ArrayType`.

  Résumé En somme, `GetArray` veille à ce que l'identificateur à accéder soit bien un tableau, que la dimension demandée par `listIndex` soit valable, et que le type final de l'accès soit cohérent avec l'environnement de types.

- `LengthArray` : la vérification contextuelle de cette classe consiste simplement à vérifier que l'identificateur rattaché à cette classe est bien un identificateur de tableau (i.e de type `ArrayType`).

4 Génération de code

Tout comme pour la vérification contextuelle. L'implémentation de l'extension **TAB** nécessite une mise à jour de certaines méthodes de génération de code, et la création d'autres méthodes implémentant la génération de code pour les nouvelles classes créées.

4.1 Modification de méthodes existantes

La première méthode à modifier est celle de la génération de code pour la déclaration de variable. En effet, il n'est pas suffisant de stocker la définition d'une variable de type **ArrayType** dans la pile. Il faut aussi stocker ses coefficients correpondants.

- `codeGenDeclVar` : Pour le cas des valeurs **simples**, la valeur est directement stockée dans la mémoire globale. Dans le cas des variables **tableaux**, un bloc est alloué pour les éléments dans le tas, et l'adresse de ce bloc est stockée dans la mémoire globale (GB).

  Dans le cas d'un tableau à plusieurs dimensions, l'allocation se fait récursivement grâce à la création de méthodes d'allocations récursives qui gèrent la création et l'initialisation de tous les niveaux intermédiaires du tableau. Cette méthode alloue des blocs mémoire pour chaque sous-tableau jusqu'à atteindre les feuilles (les coefficients du tableau). Chaque sous-tableau est ainsi représenté en mémoire comme une case mémoire qui pointe vers le sous-tableau directement inférieur. Le dernier sous-tableau pointe vers les valeurs immédiates.

  Exemple Illustratif : Initialisation d'un Tableau unidimensionnel en Mémoire

  Considérons la déclaration suivante en langage Deca :

  ```
  int b[3] = {1, 2, 3};
  ```

  Lors de cette déclaration, le compilateur procède à une allocation et une initialisation structurées de la mémoire pour le tableau `b`. Premièrement, un bloc mémoire de taille 3 est alloué dynamiquement dans le tas (`Heap`) à l'aide de l'instruction `NEW 3, R2`, où `R2` est un registre temporaire contenant l'adresse de base du tableau. Cette adresse de base (`R2`) est ensuite stockée dans la mémoire globale à un emplacement spécifique (`[GB + offset]`), assurant ainsi que le tableau `b` est correctement référencé dans la mémoire globale.

  Ensuite, chaque élément du tableau est initialisé individuellement. Par exemple, la valeur `1` est chargée dans un registre temporaire (`R3`) via `LOAD 1, R3` et ensuite stockée à l'adresse `[R2 + 0]` avec l'instruction `STORE R3, [R2 + 0]`. Ce processus est répété pour les autres éléments `b[1]` et `b[2]` respectivement.

  Dans le cas d'un tableau à plusieurs dimensions, l'allocation de mémoire est effectuée de manière récursive, chaque sous-tableau étant alloué séparément et son adresse stockée dans le tableau parent.

Exemple Illustratif : Initialisation d'un Tableau Multidimensionnel en Mémoire

Considérons la déclaration suivante en langage Deca :

```
int y[2][2] = {{4, 5}, {6, 7}};
```

Lors de l'initialisation de ce tableau à deux dimensions, le compilateur procède comme suit :

- **Allocation du Tableau Principal** : Un bloc mémoire de taille 2 (correspondant au nombre de sous-tableaux de la première dimension) est alloué dans le **tas** pour contenir les adresses des sous-tableaux. L'adresse de ce bloc est stockée dans un registre temporaire (`R2`) et référencée dans la mémoire globale (`GB`) à l'emplacement réservé pour `y`.
- **Allocation et Initialisation des Sous-Tableaux** : Pour chaque sous-tableau `y[0]` et `y[1]`, un bloc mémoire de taille 2 (correspondant au nombre d'éléments par sous-tableau) est alloué dans le tas. Les adresses de ces blocs sont stockées dans `[R2 + 0]` et `[R2 + 1]`, respectivement. Ensuite, chaque élément individuel (`y[0][0]`, `y[0][1]`, `y[1][0]`, `y[1][1]`) est initialisé en chargeant la valeur correspondante dans un registre temporaire (`R3`, `R4`) et en la stockant à l'adresse calculée.

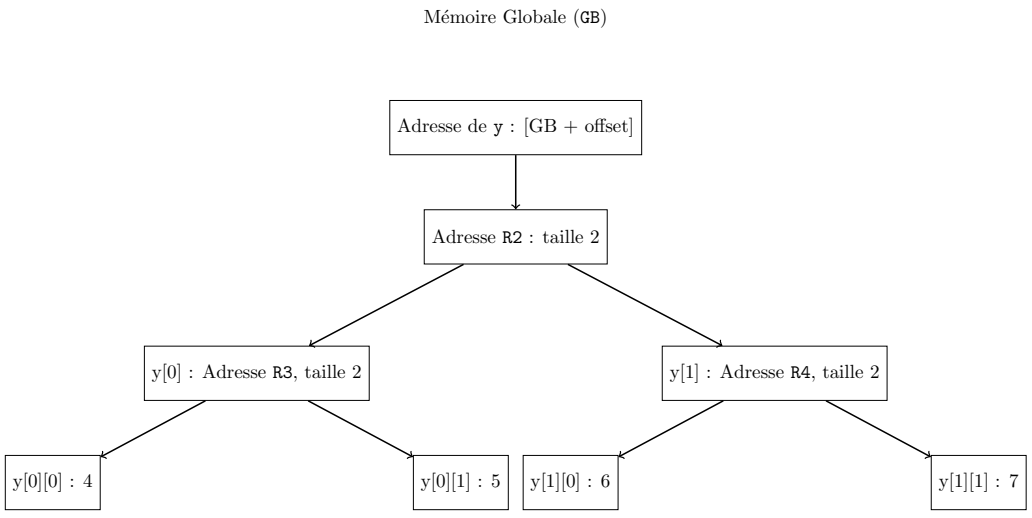


Figure 7: Disposition en mémoire lors de l'initialisation de `int y[2][2] = {{4, 5}, {6, 7}};`

Cet exemple illustre que pour un tableau multidimensionnel, chaque niveau de dimension est alloué séparément dans le tas, avec les adresses des sous-tableaux stockées dans le tableau parent.

- `codeGenInst` (classe `Assign`) : Dans le cas où l'expression à gauche est une instance de `GetArray` (cas d'un "set"), il ne s'agit pas d'une variable simple mais d'un élément de tableau, contrairement à ce qui se faisait précédemment.

  La méthode doit donc calculer l'adresse du coefficient spécifique dans le tableau à l'aide d'une nouvelle méthode `codeGenAddressOf`. Une fois l'adresse calculée, le reste du traitement s'effectue comme dans le cas d'un `Assign` normal, où la valeur de l'expression à droite est stockée à l'adresse calculée.

D'autres méthodes sont créées comme `evalueExpr`, `codeGenPrint` etc. qui reprennent la logique de ces mêmes méthodes implémentées dans le Deca Essentiel.

4.2 Implémentation de nouvelles méthodes

- **classe GetArray** :
  Cette partie présente trois méthodes implémentées pour gérer la génération de code dans la classe `GetArray`. En effet, les autres classes créées pour l'extension ne nécessitent pas de méthodes spécifiques à la génération de code. Pas d'évaluation direct d'un tableau et pas de possibilité de `print` un tableau conformément à ce qui se fait en Java.
  - `evalueExpr` : cette méthode redéfinit la méthode `evalueExpr` définie dans la classe `AbstractExpr` sert à accéder à un élément spécifique d'un tableau et est appelée lors d'un `get`. Elle consiste, comme les autres méthodes `evalueExpr`, à charger la valeur voulue dans un registre passé en paramètre.
    Son principe de fonctionnement est le suivant :
    1. On obtient l'adresse de base du tableau.
    2. Puis on parcourt chaque dimension du tableau à l'aide d'une boucle **for**, en ajoutant à chaque itération les offsets correspondant aux indices et en chargeant les adresses des pointeurs vers les tableaux intermédiaires.
    3. À la dernière dimension, la valeur chargée sera la valeur immédiate voulue.

* `codeGenPrint` : cette méthode redéfinit la méthode `codeGenPrint` définie dans la classe `AbstractExpr` et sert à print un élément d'un tableau. L'implémentation de cette méthode est similaire à celle de `evalueExpr` mais charge le résultat final dans le registre **R1** pour ensuite pouvoir être `print` avec l'instruction assembleur dédiée (suivant le type des coefficients).
  Remarque : pour que le résultat soit print, il faut d'abord vérifier que le coefficient voulu soit un coefficient terminal, i.e n'est pas de type `ArrayType`. Ceci est vérifié par la vérification contextuelle de la classe `AbstractPrint` qui renvoie une erreur le cas échéant.
* `codeGenAdressOf` : cette méthode ne redéfinit aucune méthode. Elle a été implentée dans le cas d'un "set". En effet, cette méthode renvoie l'adresse de la case mémoire qui pointe vers le coefficient voulu. Le fonctionnement est similaire aux méthodes précédentes. Son principe de fonctionnement est le suivant (en plus des étapes expliquées dans le paragraphe sur `evalueExpr`):
  1. Pour chaque dimension sauf la dernière, on charge l'adresse du sous-tableau courant dans un registre.
  2. À la dernière dimension, on obtient l'adresse finale de la case mémoire souhaitée.

  Cette méthode est utilisée lors d'une opération d'assignation (`Assign`) qui simule un "set". Elle permet de stocker la nouvelle valeur dans l'adresse mémoire récupérée par cette méthode.

– **classe LengthArray** : de la même façon que pour la classe **GetArray** (à l'exception de `codeGenAdressOf`. Des méthodes de génération de code ont été implémentées dans la classe `LengthArray` : Le fonctionnement de la méthode `evalueExpr` pour cette classe se déroule comme suit :
  1. On récupère la définition associée à l'identificateur.
  2. On accède ensuite à l'expression qui constitue l'initialisation de cette définition.
  3. La taille du tableau est déterminée selon deux approches distinctes, en fonction de la façon dont le tableau a été déclaré (*explicitement initialisé* ou *non initialisé*).

.

Remarque D'autres méthodes ont également été développées afin d'analyser entièrement les tableaux et de générer correctement leur code assembleur. Notamment les méthodes de génération de code utilisées pour la partie Deca essentiel, suivent un mécanisme proche de celui utilisé en l'absence d'objets.

5 Bibliothèque de calcul matriciel

5.1 Description des fonctionnalités

L'un des objectifs de l'extension **TAB** est de réaliser une bibliothèque de calcul matriciel. C'est-à-dire un fichier contenant un ensemble de méthodes pouvant être utilisées dans le cadre d'opérations algorithmiques ou mathématiques traitant des tableaux statiques.
Le fichier correspondant à la bibliothèque de calcul matriciel se nomme "Array.decah". C'est un fichier contenant une classe Array. Cette classe Array liste un certain nombre de méthodes permettant de faire des opérations sur des matrices. Les méthodes pouvant être utilisées dans le cadre du calcul matriciel sont les suivantes :

- `sum` : Cette méthode prend un tableau d'entiers tab et un entier len représentant la taille du tableau. Elle parcourt tous les éléments du tableau et les additionne un par un. La somme totale est ensuite retournée.

```
int sum(int[] tab, int len) {
    int i=0;
    int total = 0;
    while (i < len) {
        total = total + tab[i];
        i = i + 1;
    }
    return total;
}
```

Figure 8: méthode `sum`

- `add` : Cette méthode reçoit deux tableaux tab1 et tab2, ainsi que leurs tailles respectives len1 et len2. Si les deux tableaux n'ont pas la même taille, elle affiche un message d'erreur et retourne un tableau vide. Sinon, elle crée un nouveau tableau de même taille où chaque élément est la somme des éléments correspondants de tab1 et tab2. Ce nouveau tableau est ensuite retourné.

```
int[] add(int[] tab1, int[] tab2, int len1, int len2){
    int[] tab = new int[len1];
    int i = 0;
    int somme = 0;
    if(len1 != len2){
        println("Les tableaux ne sont pas de la même taille !");
        return tab;
    }
    else {
        while(i<len1){
            tab[i] = tab1[i] + tab2[i];
            i = i+1;
        }
    }
    return tab;
}
```

Figure 9: méthode `add`

- `sub` : De façon similaire à la méthode `add` citée précédemment, elle renvoie un nouveau tableau les différence entre les éléments du `tab1` et du `tab2` (voir figure 10).

```
int[] sub(int[] tab1, int[] tab2, int len3, int len4){
    int[] tabSous = new int[len3];
    int i = 0;
    int somme = 0;
    if(len3 != len4){
        println("Les tableaux ne sont pas de la même taille !");
        return tabSous;
    }
    else {
        while(i<len3){
            tabSous[i] = tab1[i] - tab2[i];
            i = i+1;
        }
    }
    return tabSous;
}
```

Figure 10: méthode `sub`

- `scalarMult` : Cette méthode multiplie tous les éléments d'un tableau

tab par un entier scal. Elle prend également un entier lenTab qui représente la taille du tableau. Un nouveau tableau est créé, où chaque élément est le produit de l'élément original avec le scalaire donné. Ce nouveau tableau est ensuite retourné.

```
int[] scalarMult(int[] tab, int lenTab, int scal){
    int[] tabMult = new int[lenTab];
    int i = 0;
        while(i<lenTab){
            tabMult[i] = scal * tab[i];
            i = i+1;
        }
    return tabMult;
}
```

Figure 11: méthode `scalarMult`

- `sort` : Cette méthode trie un tableau d'entiers tab en ordre croissant. La taille du tableau est donnée par lenTab. Elle utilise une version simple de l'algorithme de tri à bulles, qui compare chaque élément avec le suivant et les échange si nécessaire. Le tableau est modifié directement et n'est pas retourné.

```
void sort(int[] tab, int lenTab){
    int i = 0;
    int j;
    int temp;
    while (i < lenTab - 1) {
        j = 0;
        while (j < lenTab - i - 1) {
            if (tab[j] > tab[j + 1]) {
                temp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = temp;
            }
            j = j+1;
    }
    i = i+1;
}
}
```

Figure 12: méthode `sort`

- `toString` : Cette méthode affiche le contenu d'un tableau d'entiers tab sous la forme d'une chaîne de caractères. Elle utilise une boucle pour parcourir chaque élément du tableau, en les affichant séparés par des virgules, et encadrés par des crochets. Par exemple, pour un tableau contenant 1, 2, 3, elle affichera [1,2,3]. La taille du tableau est donnée par len.

```
void toString(int[] tab, int len) {
    int i = 0;
    print("[");
    while (i < len){
        print(tab[i]);
        if(i < (len -1)){
            print(",");
        }
        i = i +1;
    }
    println("]");
}
```

Figure 13: méthode `toString`

- `ps` : Cette méthode calcule le produit scalaire de deux tableaux d'entiers x et y. Le produit scalaire est défini comme la somme des produits des éléments correspondants des deux tableaux. Si les tableaux n'ont pas la même taille (lenx et leny), la méthode affiche un message d'erreur et retourne 0. Sinon, elle parcourt les deux tableaux simultanément, calcule le produit pour chaque paire d'éléments, additionne les résultats, et retourne la somme totale.

```
int ps(int[] x, int[] y, int lenx, int leny) {
    int i = 0;
    int res = 0;
    if(lenx != leny){
        println("Erreur, les tableaux ne sont pas de la même taille !");
        return 0;
    }
    while (i < lenx){
        res = res + (x[i]*y[i]);
        i = i+1;
    }
    return res;
}
```

Figure 14: méthode `ps`

Cette bibliothèque n'est pas exhaustive, elle peut être enrichie est maintenue à la suite de modifications ultérieures. Elle pourrait ainsi implémenter des méthodes de calculs pour des tableaux à deux dimensions par exemple.

5.2 Guide d'utilisation

Pour utiliser la bibliothèque, vous devez vous assurer d'avoir téléchargé le fichier "Array.deca" la contenant. Ensuite, vous devez l'importer en tapant la ligne suivante :

```
#include "<path-to-file>/Array.deca"
```

suivant l'endroit où le fichier a été enregistré.
Vous pouvez à présent écrire un script utilisant la bibliothèque de calcul matriciel. Pour utiliser les méthodes correspondantes, il vous suffit de créer une instance de la classe Array qui est celle contenant toutes les méthodes.

Exemple de fichier .deca utilisant la bibliothèque Ci-dessous un exemple illustrant l'utilisant de la bibliothèque pour effectuer la somme de deux tableaux

```
#include "Array.deca"

{
    Array a = new Array();
    int[] x = {8,5,6,9,7};
    int[] y = {15,2,6,8,9};
    int[] tabSomme;

    println("Tableau x : ");
    a.toString(x,lenx);

    println("Tableau y : ");
    a.toString(y,leny);

    tabSomme = a.add(x,y,lenx,leny);
    println("Somme des deux tableaux :");
    a.toString(tabSomme, lenx);
}
```

—

6 Méthodes et résultats de validation

6.1 Méthodes de validation

Pour valider nos implémentations. Nous avons suivi le même processus de validation que pour l'implémentation du langage Deca essentiel. C'est-à-dire atomiser les règles grammaticales, syntaxiques, contextuelles et de génération de code. L'objectif était d'écrire un test pour chaque règle élémentaire.
De plus, bien que ce ne soit pas réellement une méthode de validation. Nous avons mis en place des tests de non-regression pour nous assurer que les modifications ponctuelles du code n'affecte pas d'autres portions du code.

6.2 Résultats de validation

Bien que l'objectif de coller au langage Java soit plutôt bien atteint. Les tests ont fait apparaître un certain nombre de différences. Par manque de temps, il n'a pas été possible de tous les régler à temps.

- **La gestion syntaxique de la longueur d'un tableau :** la dérivation de `primary_expr` vers la règle `ident DOT LENGTH` est incomplète, c'est-à-dire que l'expression suivante :

  ```
  ...
  y[0].length;
  ...
  ```

  n'est pas reconnue syntaxiquement par notre compilateur car c'est une règle du type :

- **Adaptation des tableaux statiques dans le Deca essentiel :**

  La conception et l'architecture des tableaux statiques ont été réalisées pour être compatibles avec le Deca "sans objet". Cependant, lors des tests impliquant des opérations élémentaires effectuées à l'intérieur de classes et de méthodes, nous avons constaté un manque de gestion des tableaux dans le contexte "objet". Ces tests ont mis en évidence des lacunes dans l'architecture spécifique à l'extension, permettant ainsi de valider et d'améliorer la bibliothèque de calcul matriciel.

  De plus, l'implémentation de cette bibliothèque a servi comme une méthode de validation. En effet, son développement a permis d'identifier

des méthodes incomplètes ou non implémentées dans le Deca essentiel (par rapport au Deca sans objet). Cela a contribué à affiner et à compléter les fonctionnalités nécessaires.

C'est aussi la raison pour laquelle dans la bibliothèque de calcul matricielle, la longueur des tableaux est toujours passée en paramètre en même temps que le tableau utilisé. En effet, la génération de code de l'instruction permettant de calculer la longueur d'un tableau est possible dans le `main` mais pas dans au sein d'une méthode.
Cette prise de conscience plutôt tardive de l'adaptation de notre architecture est la raison pour laquelle la bibliothèque de calcul matriciel n'a pas été étendue à temps pour les tableaux multidimensionnels.

- **Gestion des registres :**

  Les tests impliquant des tableaux de grande dimension ont mis en évidence plusieurs problèmes liés à l'utilisation des registres. Certains registres restaient utilisés sans être correctement réinitialisés une fois qu'ils n'étaient plus nécessaires. De plus, les méthodes d'allocation de tableaux ne géraient pas de manière optimale le débordement de registres, ce qui pouvait entraîner des inefficacités ou des erreurs dans la génération de code.

  Ces observations ont permis d'identifier les lacunes et de corriger ces méthodes. Les améliorations apportées assurent désormais une meilleure gestion des registres, garantissant une utilisation plus efficace des ressources lors de la manipulation des tableaux.

7 Références bibliographiques

Voici certains liens qui nous ont permit de définir clairement l'architecture de notre compilateur, spécifié à l'extension :

- **Programmation Java/Tableaux — Wikilivres** : Introduction aux tableaux en Java, couvrant leur création, initialisation, et manipulation.
- **Les tableaux en Java — CNAM** : Document pédagogique approfondissant les notions de tableaux et leur manipulation.
- **Les tableaux en Java — Télécom SudParis** : Support de cours décrivant les bases et opérations avancées sur les tableaux.

- **Les tableaux en Java — LePPF** : Article présentant les tableaux unidimensionnels en Java avec des exemples pratiques.
- **Les tableaux en Java — TutoForYou** : Tutoriel détaillant les manipulations et fonctionnalités des tableaux en Java.